

Отчёт по лабораторной работе №7

Выполнила студентка НКАбд-02-25

Валерия Сергеевна Григорьева

Содержание

1	Цель работы	3
2	Порядок выполнения лабораторной работы	4
2.1	Реализация переходов в NASM	4
2.2	Изучение структуры файлы листинга	16
3	Задание для самостоятельной работы	19
4	Вывод	24

1 Цель работы

Цель данной работы - изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга

2 Порядок выполнения лабораторной работы

2.1 Реализация переходов в NASM

Для начала работы я создала каталог для лабораторной работы №7, перешла в него и создала файл lab7-1.asm (Рисунок 2.1).

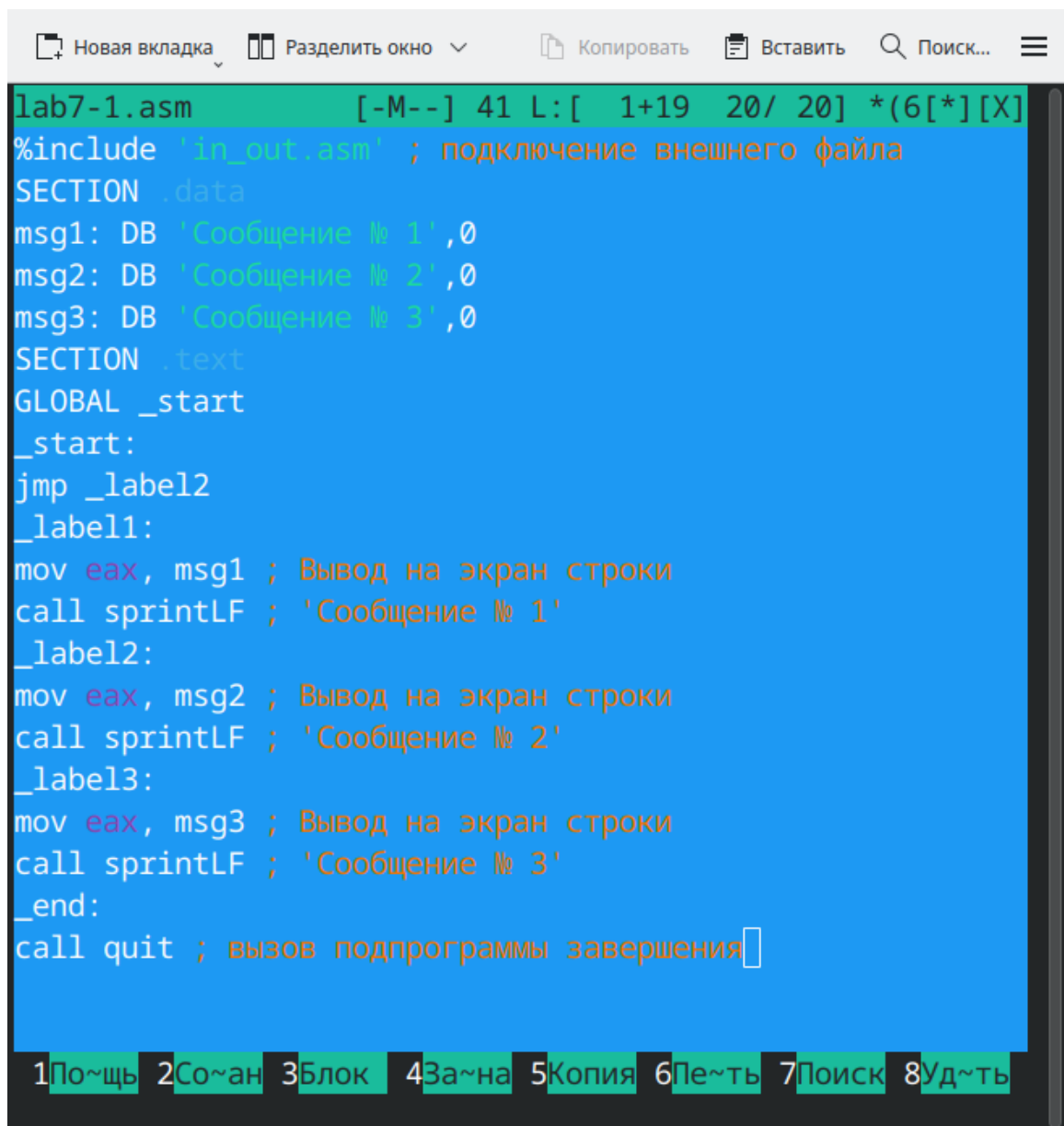
```
vsgrigoreva@fedora:~$ mkdir ~/work/arch--pc/lab07
vsgrigoreva@fedora:~$ cd ~/work/arch--pc/lab07
vsgrigoreva@fedora:~/work/arch--pc/lab07$ touch lab7-1.asm
```

Рисунок 2.1: Создание каталога и файла для работы

Затем я ввела в файл lab7-1.asm текст программы из листинга 1 (Рисунок 2.2).

```
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
```

```
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
_end:
call quit ; вызов подпрограммы завершения
```



```
lab7-1.asm [-M--] 41 L:[ 1+19 20/ 20] *(6[*][X]
#include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
_end:
call quit ; вызов подпрограммы завершения
```

1По~щъ 2Со~ан 3Блок 4За~на 5Копия 6Пе~ть 7Поиск 8Уд~ть

Рисунок 2.2: Текст программы из файла lab7-1.asm

Затем я создала исполняемый файл и запустила его (Рисунок 2.3). Для корректной работы программы скопировала файл in_out.asm в каталог с файлом lab7-1.asm.

```

vsgrigoreva@fedora:~/work/arch--pc/lab07$ nasm -f elf lab7-1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 3
vsgrigoreva@fedora:~/work/arch--pc/lab07$ █

```

Рисунок 2.3: Создание исполняемого файла и запуск программы

Программа корректно вывела значения и завершила работу.

Использование инструкции `jmp _label2` меняет порядок исполнения инструкций и позволяет выполнить инструкции начиная с метки `_label2`, пропустив вывод первого сообщения.

Инструкция `jmp` позволяет осуществлять переходы не только вперед но и назад. Далее я изменила программу таким образом, чтобы она выводила сначала „Сообщение № 2“, потом „Сообщение № 1“ и завершала работу (Рисунок 2.4).

```

#include 'in_out.asm' ; подключение внешнего файла

SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0

SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end
_label2:
mov eax, msg2 ; Вывод на экран строки

```

```
call sprintf ; 'Сообщение № 2'
jmp _label1
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
_end:
call quit ; вызов подпрограммы завершения
```



```
lab7-1.asm      [-M--]  8 L:[ 1+12 1[*][X]
#include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
jmp _label1
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
_end:
call quit ; вызов подпрограммы завершения
```

1Поиск 2Сохран 3Блок 4Замена 5Копия 6Печать

Рисунок 2.4: Измененный текст программы

Затем я создала исполняемый файл и проверила его работу (Рисунок 2.5).

```

vsgrigoreva@fedora:~/work/arch--pc/lab07$ nasm -f elf lab7-1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 1

```

Рисунок 2.5: Запуск программы

Программ корректно вывела сообщения и завершила работу.

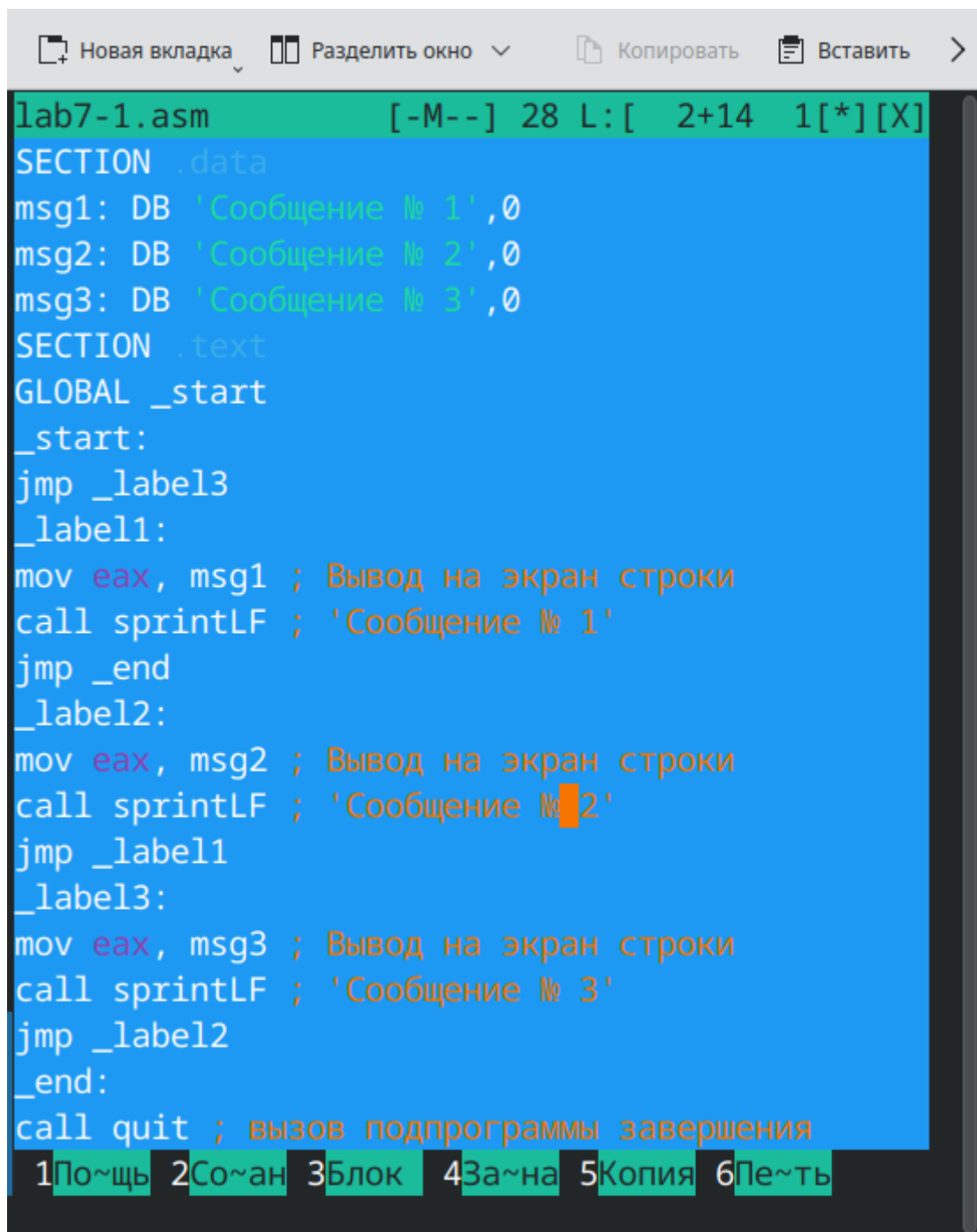
Затем я изменила текст программы таким образом, чтобы она выводила сначала Сообщение №3, потом №2 и №1 (Рисунок 2.6).

```

#include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label3
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
jmp _label1
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'

```

```
jmp _label2  
_end:  
call quit ; вызов подпрограммы завершения
```



```
lab7-1.asm [-M--] 28 L:[ 2+14 1[*][X]
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label3
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
jmp _label1
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
jmp _label2
_end:
call quit ; вызов подпрограммы завершения
1По~щъ 2Со~ан 3Блок 4За~на 5Копия 6Пе~ть
```

Рисунок 2.6: Измененный текст программы

Далее я создала исполняемый файл и запустила программу (Рисунок 2.7). Программа корректно выполнила работу.

```

vsgrigoreva@fedora:~/work/arch--pc/lab07$ nasm -f elf lab7-1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1

```

Рисунок 2.7: Запуск программы

Часто при написании программ необходимо использовать условные переходы, т.е. переход должен происходить если выполнено какое-либо условие. Я создала файл lab7-2.asm в каталоге ~/work/arch-pc/lab07 (Рисунок 2.8) и ввела в него текст из листинга 4 (Рисунок 2.9).

```

vsgrigoreva@fedora:~/work/arch--pc/lab07$ touch lab7-2.asm
vsgrigoreva@fedora:~/work/arch--pc/lab07$ 

```

Рисунок 2.8: Создание файла

```

%include 'in_out.asm'

section .data
msg1 db 'Введите B: ',0h
msg2 db "Наибольшее число: ",0h
A dd '20'
C dd '50'

section .bss
max resb 10
B resb 10

section .text
global _start
_start:
; ----- Вывод сообщения 'Введите B: '
mov eax,msg1

```

```

call sprint
; ----- Ввод 'B'
mov ecx,B
mov edx,10
call sread
; ----- Преобразование 'B' из символа в число
mov eax,B
call atoi ; Вызов подпрограммы перевода символа в число
mov [B],eax ; запись преобразованного числа в 'B'
; ----- Записываем 'A' в переменную 'max'
mov ecx,[A] ; 'ecx = A'
mov [max],ecx ; 'max = A'
; ----- Сравниваем 'A' и 'C' (как символы)
cmp ecx,[C] ; Сравниваем 'A' и 'C'
jg check_B ; если 'A>C', то переход на метку 'check_B',
mov ecx,[C] ; иначе 'ecx = C'
mov [max],ecx ; 'max = C'
; ----- Преобразование 'max(A,C)' из символа в число
check_B:
mov eax,max
call atoi ; Вызов подпрограммы перевода символа в число
mov [max],eax ; запись преобразованного числа в 'max'
; ----- Сравниваем 'max(A,C)' и 'B' (как числа)
mov ecx,[max]
cmp ecx,[B] ; Сравниваем 'max(A,C)' и 'B'
jg fin ; если 'max(A,C)>B', то переход на 'fin',
mov ecx,[B] ; иначе 'ecx = B'
mov [max],ecx

```

```

; ----- Вывод результата
fin:
mov eax, msg2
call sprint ; Вывод сообщения 'Наибольшее число: '
mov eax, [max]
call iprintLF ; Вывод 'max(A,B,C)'
call quit ; Выход

```

```

lab7-2.asm [-M--] 17 L:[ 1+ 0 1/ 49] *(17 /1743b) 00[*][X]
#include 'in_out.asm'
section .data
msg1 db 'Введите B: ',0h
msg2 db "Наибольшее число: ",0h
A dd '20'
C dd '50'
section .bss
max resb 10
B resb 10
section .text
global _start
_start:
; ----- Вывод сообщения 'Введите B: '
mov eax,msg1
call sprint
; ----- Ввод 'B'
mov ecx,B
mov edx,10
call sread
; ----- Преобразование 'B' из символа в число
mov eax,B
call atoi ; Вызов подпрограммы перевода символа в число

```

1По~щъ 2Со~ан 3Блок 4За~на 5Копия 6Пе~ть 7Поиск 8Уд~ть 9Ме~мс 10Выход

Рисунок 2.9: Текст программы

Затем я создала исполняемый файл и проверила работу для нескольких значений B (Рисунок 2.10).

```

vsgrigoreva@fedora:~/work/arch--pc/lab07$ nasm -f elf lab7-2.asm
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ./lab7-2
Введите В: 19
Наибольшее число: 50
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ./lab7-2
Введите В: 54
Наибольшее число: 54
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ./lab7-2
Введите В: 21
Наибольшее число: 50
vsgrigoreva@fedora:~/work/arch--pc/lab07$ 

```

Рисунок 2.10: Создание исполняемого файла и запуск программы

Программа корректно вывела значения для всех введенных В и завершила работу.

2.2 Изучение структуры файлы листинга

Обычно `nasm` создаёт в результате ассемблирования только объектный файл. Получить файл листинга можно, указав ключ `-l` и задав имя файла листинга в командной строке. Я создала файл листинга для программы из файла `lab7-2.asm` (Рисунок 2.11).

```

vsgrigoreva@fedora:~/work/arch--pc/lab07$ nasm -f elf -l lab7-2.lst lab7-2.asm
vsgrigoreva@fedora:~/work/arch--pc/lab07$ 

```

Рисунок 2.11: Создание файла листинга

Затем я открыла файл листинга `lab7-2.lst` с помощью `mcedit` (Рисунок 2.12).


```
lab7-2.lst [----] 0 L: [ 1+ 0 1/225] *(0 /14458b) 0032 0x020 [*][X]
1 %include 'in_out.asm'
2 <1> ;----- slen -----
3 <1> ; Функция вычисления длины сообщения
4 <1> slen:
5 <1> push ebx
6 <1> mov ebx, eax
7 <1>
8 <1> nextchar:
9 <1> cmp byte [eax], 0
10 <1> jz finished
11 <1> inc eax
12 <1> jmp nextchar
13 <1>
14 <1> finished:
15 <1> sub eax, ebx
16 <1> pop ebx
17 <1> ret
18 <1>
19 <1> ;----- sprint -----
20 <1> ; Функция печати сообщения
21 <1> ; входные данные: mov eax, <message>
1Помощь 2Сохран 3Блок 4Замена 5Копия 6Пере-ить 7Поиск 8Удалить 9МенюМС 10Выход
```

Рисунок 2.12: Текст файла листинга

Я внимательно ознакомилась с его форматом и содержимым.

Например, в строке „00000003 803800 <1> cmp byte [eax], 0“ инструкция сравнивает байт по адресу, хранящемуся в регистре EAX, с нулём. Используется для проверки конца строки в функции вычисления длины строки. Машинный код 80 38 00 представляет команду `cmp r/m8, imm8`.

В строке „00000013 E8E8FFFFFF <1> call slen“ команда `call` вызывает подпрограмму `slen`, передавая управление по относительно смещению, указанному в машинном коде E8 E8 FF FF FF. Перед переходом процессор сохраняет адрес возврата в стек. Эта функция используется для вычисления длины строки перед её выводом.

Строка „00000000 D092D0B2D0B5D0B4D0- msg1 db 'Введите В:“,0h“ определяет в сегменте `.data` текстовое сообщение «Введите В: », закодированное в UTF-8, что видно из двубайтных последовательностей вида D0 92, D0 B2 и т. д. Директива `db` размещает эти байты в памяти, а `0h` добавляет нулевой терминатор для корректной работы функций обработки строк.

Затем я открыла файл с программой `lab7-2.asm` и в одной инструкции удалила один операнд. Выполнила трансляцию с получением файла листинга (Рисунок 2.13).

```
vsgrigoreva@fedora:~/work/arch--pc/lab07$ nasm -f elf -l lab7-2.lst lab7-2.asm
lab7-2.asm:14: error: invalid combination of opcode and operands
vsgrigoreva@fedora:~/work/arch--pc/lab07$
```

Рисунок 2.13: Трансляция с получением файла листинга

В этом случае создается файл с листингом lab7-2.lst, при этом в тексте листинга в строке, из которой был удален операнд, появляется ошибка (Рисунок 2.14).

```
12          _start:
13          ; ----- Вывод сообщения 'Введите B: '
14          mov eax
14          *****
15 000000E8 E822FFFFFF      error: invalid combination of opcode and operands
15          call sprint
16          ; ----- Ввод 'B'
17 000000ED B9[0A000000]      mov ecx,B
18 000000F2 BA0A000000      mov edx,10
19 000000F7 E847FFFFFF      call sread
```

Рисунок 2.14: Ошибка в тексте листинга

3 Задание для самостоятельной работы

1. Написала программу нахождения наименьшей из 3 целочисленных переменных a, b, c. Значения переменных выбрала из таблицы в соответствии с вариантом, полученным в предыдущей лабораторной работе (вар. 15 a=32, b=6, c=54).

```
%include 'in_out.asm'

SECTION .data
A dd 32
B dd 6
C dd 54
msg DB 'Наименьшее из A,B,C: ',0

SECTION .bss
minv RESD 1

SECTION .text
GLOBAL _start

_start:
    ; Сначала min = A
```

```

    mov eax, [A]
    mov [minv], eax

; сравнение min и B
    cmp eax, [B]
    jle check_C
    mov eax, [B]
    mov [minv], eax

check_C:
    cmp eax, [C]
    jle print_min
    mov eax, [C]
    mov [minv], eax

print_min:
    mov eax, msg
    call sprint
    mov eax, [minv]
    call iprintLF

    call quit

```

Затем создала исполняемый файл и проверила его работу (Рисунок 2.15).

```

vsgrigoreva@fedora:~/work/arch--pc/lab07$ touch sam1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab07$ nasm -f elf sam1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ld -m elf_i386 -o sam1 sam1.o
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ./sam1
Наименьшее из A,B,C: 6
vsgrigoreva@fedora:~/work/arch--pc/lab07$

```

Рисунок 3.1: Создание исполняемого файла и запуск программы

Программа корректно вывела наименьшее значение.

2. Я написала программу, которая для введенных с клавиатуры значений x и a вычисляет значение заданной функции $f(x)$ и выводит результат вычислений. Вид функции $f(x)$ выбрала в соответствии с вариантом, полученным в предыдущей лабораторной работе ($x < a$ и $x \geq a$).

```
%include 'in_out.asm'

SECTION .data
msg_x    DB 'Введите x: ',0
msg_a    DB 'Введите a: ',0
msg_res  DB 'Результат f(x): ',0

SECTION .bss
bufx     RESB 80
bufa     RESB 80
valx     RESD 1
vala     RESD 1

SECTION .text
GLOBAL _start

_start:
    ; Ввод x
    mov eax, msg_x
    call sprint
    mov ecx, bufx
    mov edx, 80
```

```

call sread

; Ввод a
mov eax, msg_a
call sprint
mov ecx, bufa
mov edx, 80
call sread

; преобразование x -> число
mov eax, bufx
call atoi
mov [valx], eax

; преобразование a -> число
mov eax, bufa
call atoi
mov [vala], eax

; условие: if x < a -> result = a + 10
mov eax, [valx]      ; eax = x
cmp eax, [vala]      ; compare x, a
jnl less_than_a

; ветвь x >= a: result = x + 10
mov eax, [valx]
add eax, 10
jmp print_result

```

```

less_than_a:
    mov eax, [vala]
    add eax, 10

print_result:
    ; печать результата
    mov edi, eax
    mov eax, msg_res
    call sprint
    mov eax, edi
    call iprintLF

    call quit

```

Далее я создала исполняемый файл и проверила его работу для значений `x` и `a` (Рисунок 2.16).

```

vsgrigoreva@fedora:~/work/arch--pc/lab07$ nasm -f elf sam2.asm
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ld -m elf_i386 -o sam2 sam2.o
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ./sam2
Введите x: 2
Введите a: 3
Результат f(x): 13
vsgrigoreva@fedora:~/work/arch--pc/lab07$ ./sam2
Введите x: 4
Введите a: 2
Результат f(x): 14
vsgrigoreva@fedora:~/work/arch--pc/lab07$ 

```

Рисунок 3.2: Создание исполняемого файла и запуск программы

Программа корректно вывела результаты для разных значений `x` и `a`.

4 Вывод

В ходе выполнения работы я изучила команды условного и безусловного переходов, приобрела навыки написания программ с использованием переходов, познакомилась с назначением и структурой файла листинга.